



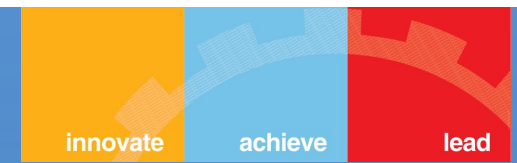
BITS Pilani

Conversational AI

RLHF-PEFT

Dr Bharathi R

Adjunct Professor, CSIS WILP BITS PILANI



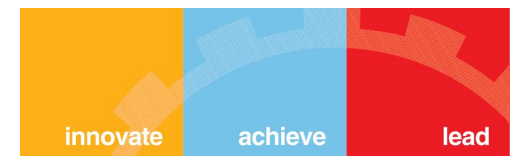
BITS Pilani



AIMLCZG521 , Conversational AI

Lecture No. 7 | Module 2

Disclaimer & Acknowledgement



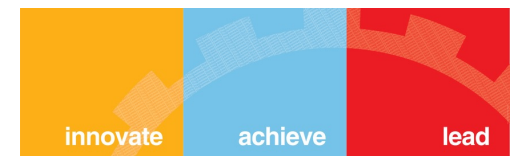
DISCLAIMER

These instructional materials have been developed by the instructor for educational purposes. All content is intended solely to support the learning objectives of this course. While every effort has been made to ensure accuracy and completeness, learners are encouraged to consult original source publications for authoritative reference.

ACKNOWLEDGEMENT

These instructional materials have been developed by the instructor with due acknowledgement of the original publication authors and the broader academic community for their openly accessible resources, along with special recognition of Prof. Bhagath's Conv-AI-WILP (October 2025) presentation materials, which have been adapted and modified to support the learning objectives of this course.

Session 7: Outline



RLHF

1. Post-Training Overview

SFT vs RLHF approaches

2. Reinforcement Learning Basics

Agent, State, Action, Reward

3. InstructGPT & PPO

Reward model training, PPO fine-tuning

4. Direct Preference Optimization (DPO)

Eliminating the reward model

5. GRPO

Group Relative Policy Optimization

6. RLHF Limitations

Open problems & challenges

7. State of the Art (SoTA)

Recent advances in alignment

PEFT

1. Full FT vs PEFT

2. Why PEFT exists: A key trade-off

3. Prompt & Prefix Tuning

Soft prompt methods (Lester 2021, Li 2021)

4. Adapter-Based Methods

Injecting small trainable modules (Houlsby 2019)

5. LoRA

Low-rank reparameterization (Hu 2021)

6. Quantization & QLoRA

4-bit NF4, double quant, paged optimizers (Dettmers 2023)

Where Post-Training(Fine tuning and RL) Fits ?



Post training

Fine-tuning (Supervised Fine-Tuning - SFT): This approach involves training the model on a dataset of curated examples, including harmful questions paired with safe answers.

Input: "How do I make a dangerous weapon at home?"

Target output: "I cannot provide instructions for creating harmful devices."

Reinforcement Learning (RL) / RLHF: This method involves an AI agent learning by interacting with an environment and receiving feedback, typically in the form of rewards or penalties, to optimize its behavior.

Input: "How do I make a dangerous weapon at home?"

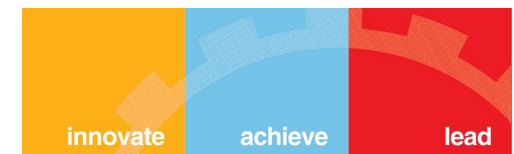
Response A:
"Here is how to make a dangerous weapon..."

*penalized (negative
"Reward [score]")*

Response B: "I cannot provide instructions..."

*rewarded,
guiding the model
towards the safer
behavior.*

Post Training



Step 1

Collect demonstration data, and train a supervised policy.

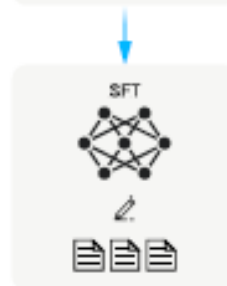
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

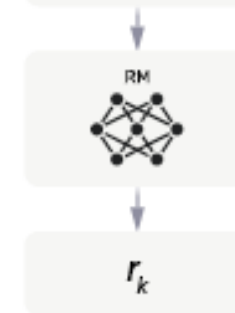
A new prompt is sampled from the dataset.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.

SFT-Supervised fine-tuning

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

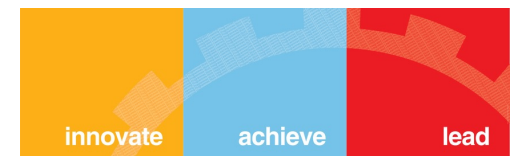
A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3 with supervised learning.



Prompt (Input)	Target Response (Output)	Task Type	Notes
What is gravity?	Gravity is a fundamental force that causes objects with mass to attract each other.	Factual QA	Teaches concise, correct explanation
Solve: 12×7	$12 \times 7 = 84$	Math	Simple deterministic answer
Translate to French: "The sky is blue."	Le ciel est bleu.	Translation	Language mapping
Summarize: "Photosynthesis is the process by which plants..."	Photosynthesis is how plants convert light into chemical energy.	Summarization	Output compression
Write a polite email declining a meeting.	Thank you for the invitation. Unfortunately, I won't be able to attend...	Style / Tone	Teaches politeness
Classify sentiment: "I love this product."	Positive	Classification	Label-style output
Explain this code snippet.	This code initializes a loop that iterates over the list...	Code explanation	Instruction following
Answer step by step: What is 15% of 200?	$15\% \text{ of } 200 = 0.15 \times 200 = 30$. Answer: 30	Reasoning	Teaches structured reasoning

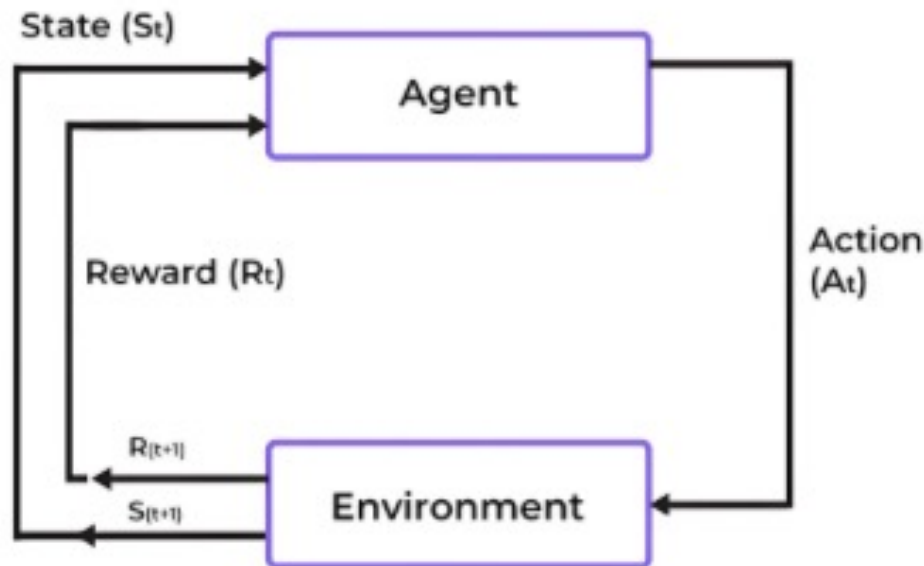
Common Objectives of Learning from Human Feedback (RLHF)



- **Alignment:** Ensuring the model's output reflects human values and intentions.
- **Trust & Factuality:** Making the model reliable and accurate in the information it provides.
- **Fairness:** Addressing social biases to ensure the model treats all demographics equitably.
- **Explainability:** Ensuring the model can provide clear, logical reasons for its responses.

Reinforcement Learning (RL) Model

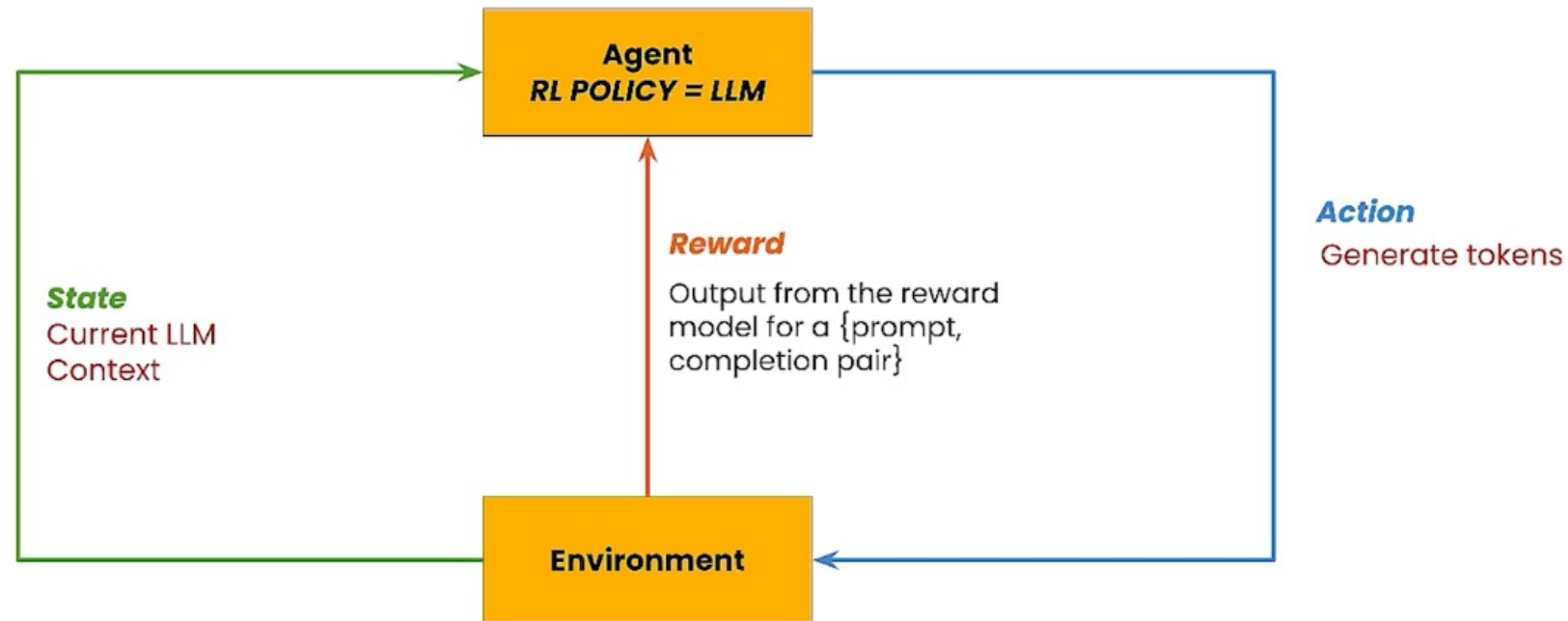
REINFORCEMENT LEARNING MODEL



Core Components

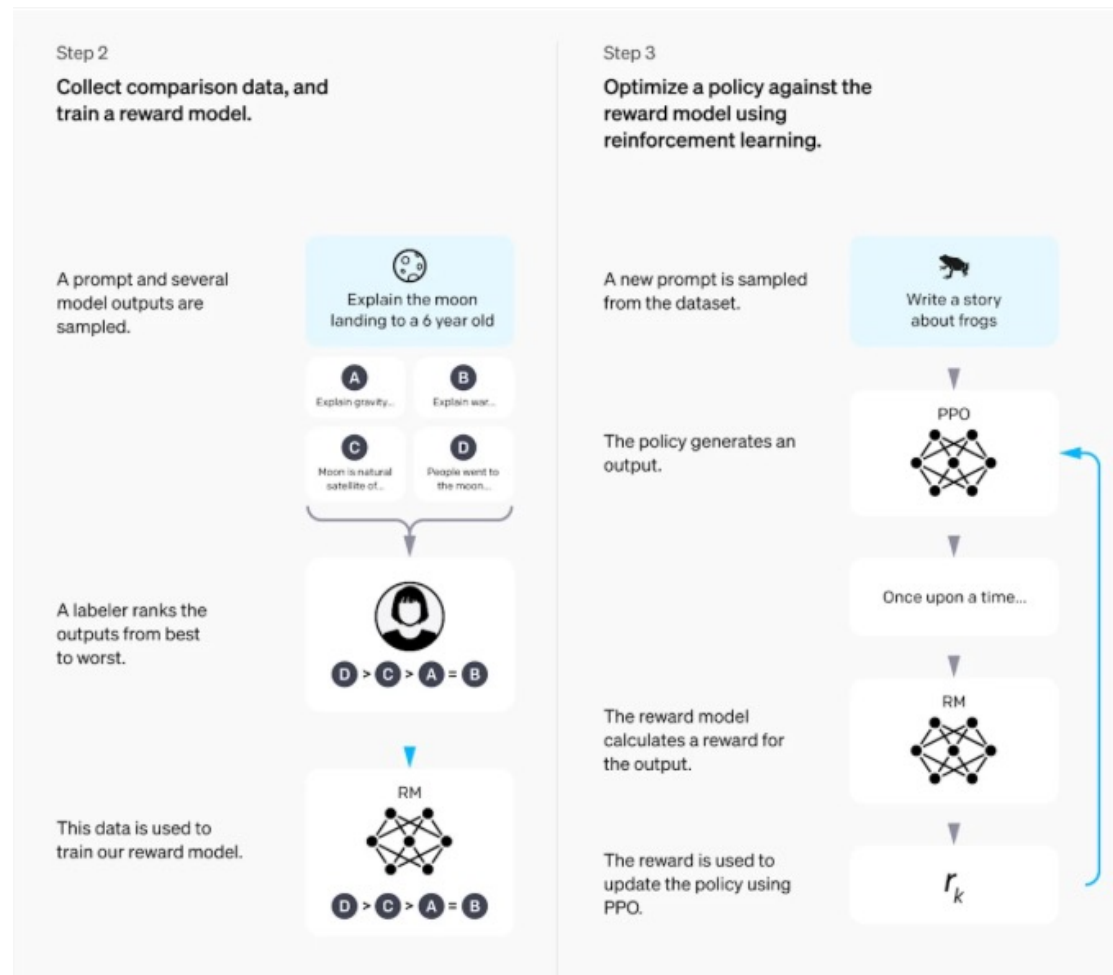
- **Agent:** The learner or decision-maker that takes actions.
- **Environment:** The external system the agent interacts with.
- **State S_t :** The current situation or configuration of the environment at time t .
- **Action A_t :** The choice made by the agent based on the current state.
- **Reward R_t :** Numerical feedback received from the environment as a result of an action.

RL terms translated to LLM terms



Proximal Policy Optimization (PPO)

InstructGPT: Training language models to follow instructions with human feedback. (Ouyang et. al, 2022)



- **Agent:** language model
- **Action:** predict the next token
- **Policy :** the output distribution of the next token
- **Reward :** a reward model trained by human evaluations on model responses, so no more human-in-the-loop is needed

Instruct GPT : Step 2 and Step 3

Feature	Step 2 (Train the Judge)	Step 3 (Train the Student)
Primary Output	Produces a Reward Model (RM) .	Produces the final Policy (Language Model) .
Source of Feedback	Humans serve as the feedback mechanism. They manually look at outputs and rank them.	The Reward Model serves as the feedback mechanism. It calculates rewards automatically.
Role of Humans	Human-in-the-loop: Humans are actively required to label and rank data.	No human-in-the-loop: The Reward Model replaces the human, allowing the AI to train on a massive scale without needing a person to grade every single sentence.
Data Type	Uses Comparison Data (Ranking A vs B vs C).	Uses Scalar Rewards (A numerical score calculated by the RM) to update the policy.

Step 2
Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.

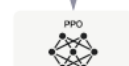


Step 3
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



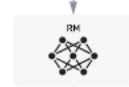
The policy generates an output.



Once upon a time...



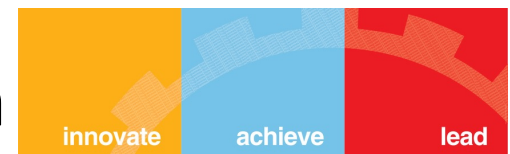
The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Reward Model (RM) Training, a critical step in Reinforcement Learning from Human Feedback (RLHF).



Step 2

Collect comparison data,
and train a reward model.

A prompt and
several model
outputs are
sampled.



A labeler ranks
the outputs from
best to worst.

This data is used
to train our
reward model.

- Prompt supervised fine-tuned language model with to produce pairs of answers

$$(y_1, y_2) \sim \pi^{\text{SFT}}(y | x)$$

- Human annotators decide which one wins / is preferred

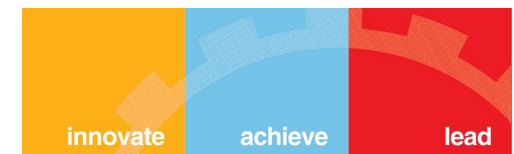
$$y_w > y_l | x$$

- A reward model is trained to score y_w higher than y_l

$$\mathcal{L}_R(r_\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))] \quad \text{Eq.1}$$

- A reward model is often initialized from π^{SFT} with a linear layer to produce a scalar reward value
- σ : The sigmoid function.
- $r_\phi(x, y)$: The reward score assigned by the model to a given prompt-response pair.
- **Goal:** Maximize the difference between the reward for the winning response (y_w) and the losing response (y_l).

Fine-Tuning with RL: PPO



- Optimize the language model π_θ with feedback from the reward model **Objective Function**:

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)] - \beta \mathbb{D}_{\text{KL}} [\pi_\theta(y | x) || \pi_{\text{ref}}(y | x)]$$

Diagram illustrating the components of the PPO objective function:

- The first term, $\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(y|x)} [r_\phi(x, y)]$, is associated with the box: "prefer responses with high rewards".
- The second term, $\beta \mathbb{D}_{\text{KL}} [\pi_\theta(y | x) || \pi_{\text{ref}}(y | x)]$, is associated with the box: "control the deviation from the reference policy, the π^{SFT} model".

Step 3
Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017.

Fine-Tuning with RL: PPO

Objective Function:



$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(y|x)} [r_{\phi}(x, y)] - \beta \mathbb{D}_{\text{KL}} [\pi_{\theta}(y | x) \parallel \pi_{\text{ref}}(y | x)]$$

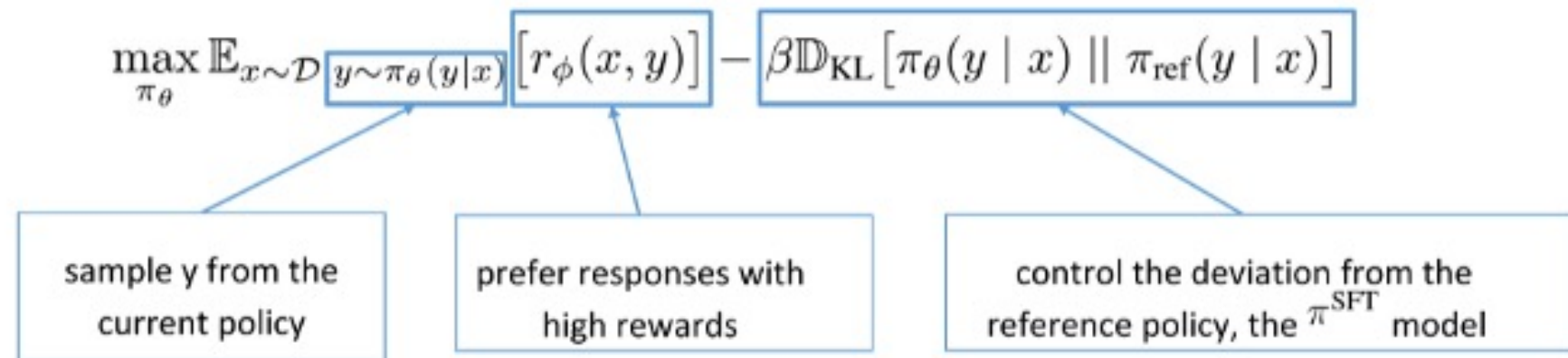
Diagram illustrating the components of the PPO objective function:

- prefer responses with high rewards** (points to $r_{\phi}(x, y)$)
- control the deviation from the reference policy, the π^{SFT} model** (points to $\pi_{\text{ref}}(y | x)$)

- **Reward Maximization:** $\mathbb{E}[r_{\phi}(x, y)]$
 - **Goal:** Direct the model to generate responses (y) that receive high scores from the reward model (r_{ϕ}).
- **KL Divergence Penalty:** $-\beta \mathbb{D}_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}]$
 - **Goal:** Penalize the model if it drifts too far from the original reference model (π_{ref} or π^{SFT}).
 - **β :** A hyperparameter that controls the strength of this penalty.

Fine-Tuning with RL: PPO

- Optimize the language model π_θ with feedback from the reward model r_ϕ



- prevent mode-collapse to single high reward answers
- prevent the model deviating too far from the distribution where the reward model is accurate

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017.

Fine-Tuning with RL: PPO



- Training objective

$$E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} [r_{\theta}(x, y) - \beta \log (\pi_{\theta}(y | x) / \pi_{\text{ref}}(y | x))] + \\ \gamma E_{x \sim D_{\text{pretrain}}} [\log(\pi_{\theta}(x))]$$

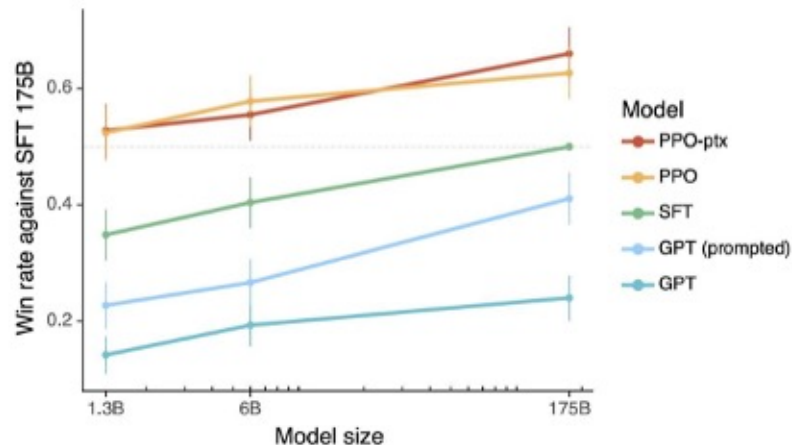
- Add pre-training gradients to fix the performance regressions on public NLP tasks
- For PPO models, γ is set to 0

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017.

Comparison with Baselines



- RLHF models are more preferred by human labelers

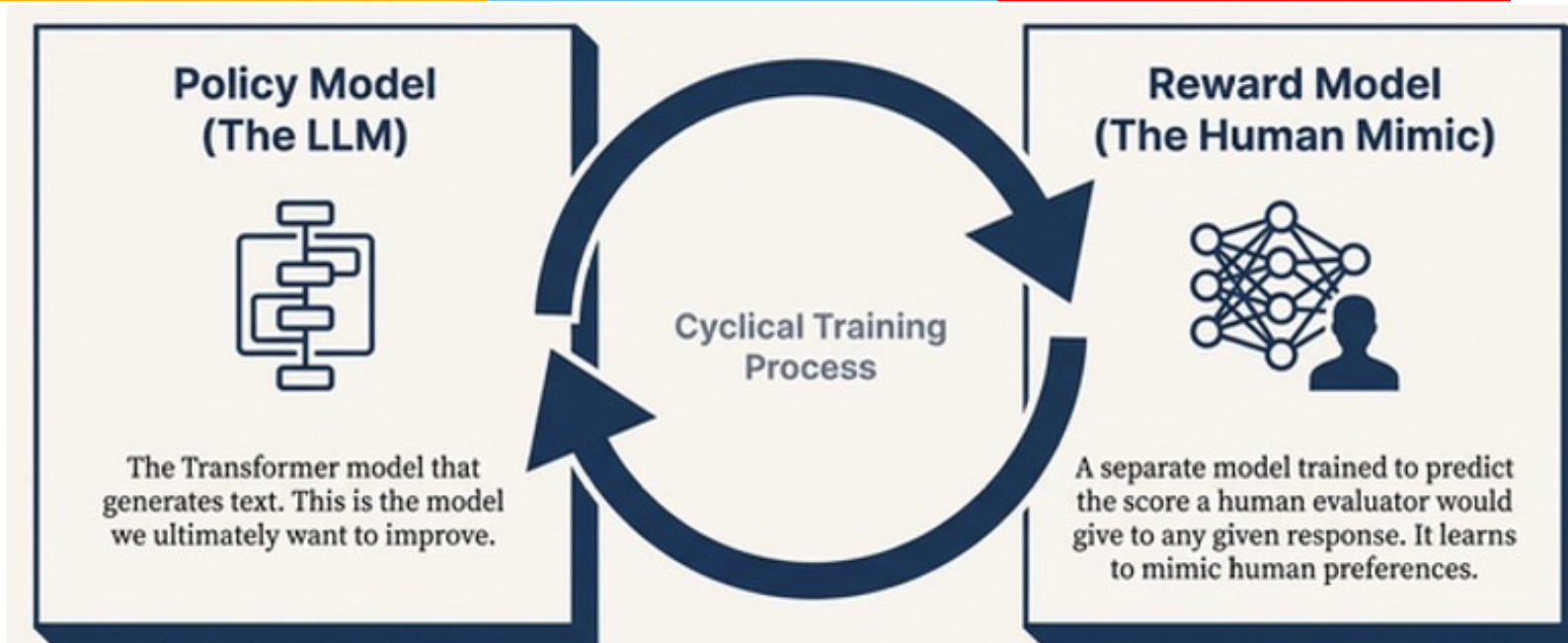


Human evaluations of various models on our API prompt distribution, evaluated by how often outputs from each model were preferred to those from the 175B SFT model. Our InstructGPT models (PPO-ptx) as well as its variant trained without pretraining mix (PPO) significantly outperform the GPT-3 baselines (GPT, GPT prompted); outputs from our 1.3B PPO-ptx model are preferred to those from the 175B GPT-3. Error bars throughout the paper are 95% confidence intervals.

Limitation of PPO methods

- Need to train multiple models: a reward model and a policy model
- Need sampling from LM during fine-tuning
- The RL training process is too complicated!
- Is it possible to directly train a language model from the human preference annotations?

The RLHF Architecture – A Two model Dance



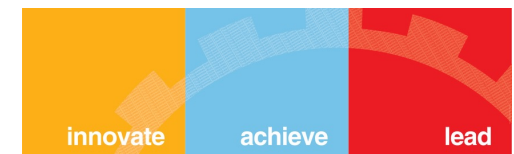
The Cyclical Training Process

The two models refine each other in a loop:

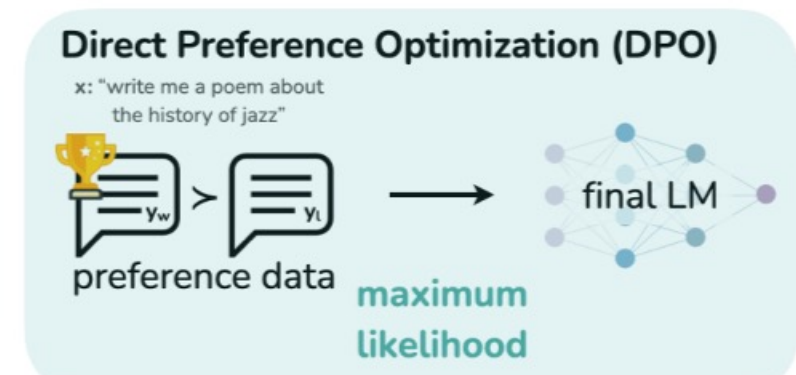
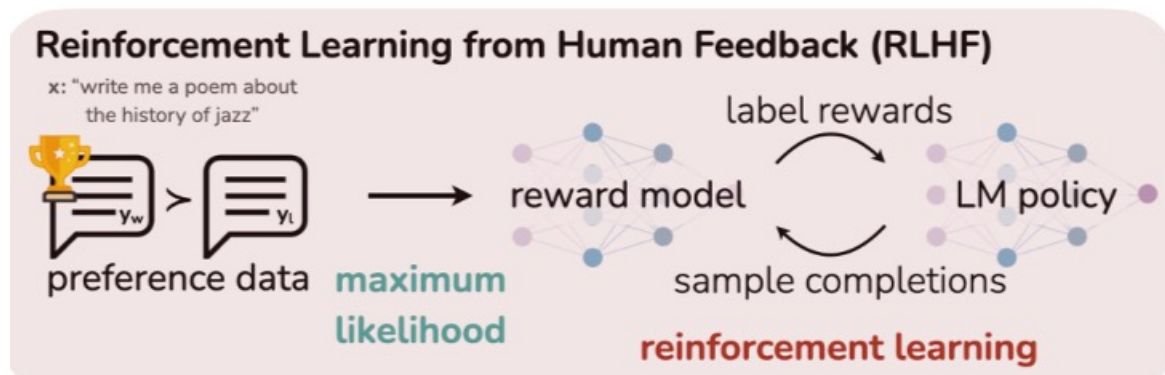
- **Feedback Signal:** The Reward Model provides a scoring signal that tells the Policy Model which of its generated responses are better.
- **Model Optimization:** The Policy Model uses this feedback (often via an algorithm like PPO) to update its internal parameters.
- **Data Refinement:** As the Policy Model generates new, increasingly sophisticated data, that data can be used to further train and sharpen the Reward Model's accuracy.

DPO: Direct Preference Optimization:

Your Language Model is Secretly a Reward Model

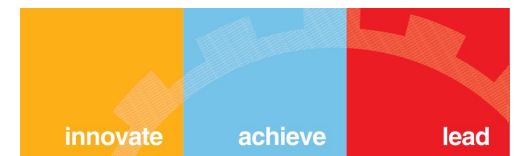


Cutting out the middleman in LLM alignment w/ Direct Policy Optimization (DPO), compared to PPO.



(Rafailov et al., 2023)

How many models? (In DPO)

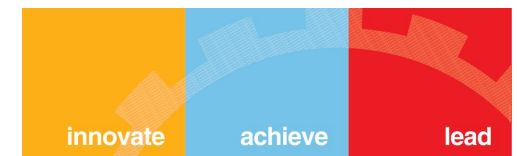


- 1.The Active Policy (π_{θ}):** The model being trained (the "Student").
- 2.The Reference Policy (π_{ref}):** A frozen copy of the original model (usually the Supervised Fine-Tuned model).

Why? Even though there is no separate "Reward Model," DPO still needs this reference model to calculate the implicit reward and ensure the new model doesn't drift too far from valid language (using KL divergence).

Key Takeaway: DPO **removes** the separate Reward Model (r_{ϕ}) and Value Model/Critic often used in PPO, reducing the memory burden significantly.

Direct Preference Optimization



- Recall the reward model training loss

Recall the Reward model training loss **Eq-1**

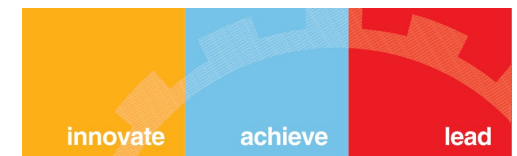
$$\mathcal{L}_R(r_\phi, \mathcal{D}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$$

- DPO training objective:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

- A simple classification loss!

DPO Training Objective (Main Equation)

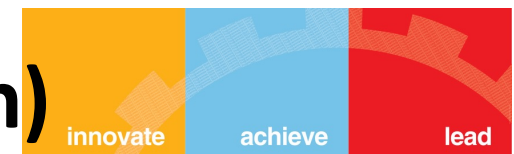


$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

DPO simplifies this by removing the middleman.

- **Core Insight:** The authors discovered a mathematical relationship showing that "Your Language Model is Secretly a Reward Model".
- **The Process:** Instead of training a separate Reward Model, DPO defines the loss function **directly** on the language model itself using the preference data.
- **Benefit:** It bypasses the need for sampling and the separate Reward Model. It treats the problem as a **simple classification task** (classifying which response is "better") rather than a reinforcement learning task.

DPO Training Objective (Main Equation)



$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right]$$

1. The Goal:

The objective is to maximize the probability of the **winning answer** (y_w) and minimize the probability of the **losing answer** (y_l).

2. The Ratio $\frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)}$:

This ratio measures the extent to which the new model π_{θ} deviates from the reference model π_{ref} .

- A **high ratio** is desired for the winning response, indicating that the new model assigns it higher likelihood than the reference model.
- A **low ratio** is desired for the losing response, indicating that the new model assigns it lower likelihood than the reference model.

3. Beta (β):

This parameter functions as a temperature control that regulates how strongly the model is allowed to deviate from the reference policy.

4. Sigmoid (σ) and Logarithm:

These functions transform the values into probabilities between 0 and 1 and ensure smooth, stable optimization.

DPO -Numericals

Prompt

“Explain gravity simply.”

Responses

Preferred (y_v): “Gravity pulls objects toward each other.”

Rejected (y_l): “Gravity is caused by air pressure.”

For a given prompt, the model probabilities are:

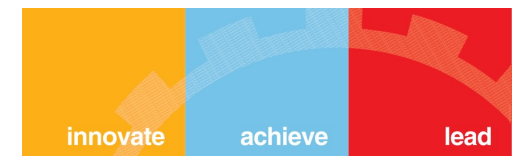
Assume $\beta=1$.

(a) Compute the DPO loss step by step.

(b) Briefly interpret what the resulting loss value indicates about the model’s behavior.

Model	$P(y_w x)$	$P(y_l x)$
Reference model (π_{ref})	0.40	0.30
Current model (π_θ)	0.60	0.10

DPO Numericals



Step 1: Compute Likelihood Ratios

For the preferred response:

$$\frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} = \frac{0.60}{0.40} = 1.5$$

For the rejected response:

$$\frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} = \frac{0.10}{0.30} \approx 0.333$$

Step 2: Compute Log Ratios

$$\log(1.5) \approx 0.405$$

$$\log(0.333) \approx -1.099$$

Step 3: Compute Difference of Log Ratios

$$0.405 - (-1.099) = 1.504$$

Step 4: Apply β and Sigmoid

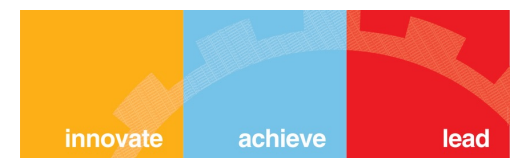
Since $\beta = 1$:

$$\sigma(1.504) = \frac{1}{1 + e^{-1.504}} \approx 0.818$$

Step 5: Compute Final DPO Loss

$$\mathcal{L}_{\text{DPO}} = -\log(0.818) \approx 0.20$$

This value represents how much more the current model prefers the winning response over the losing response, relative to the reference model.



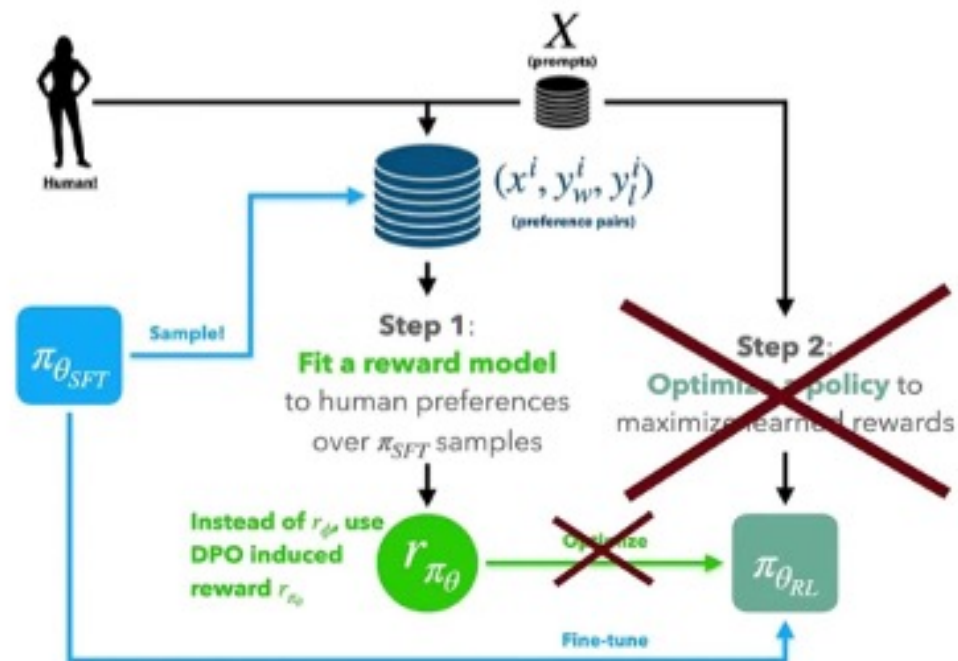
The magic of DPO:

1. Usually, you need a separate neural network to calculate the Reward (r).
2. But this derivation proves that **the optimal policy itself contains the reward information**.
3. Therefore, we can delete the Reward Model entirely and just define the reward *implicitly* using the ratio between the trained model and the reference model.

This derivation justifies why DPO works: it mathematically proves that optimizing preferences directly is equivalent to training a Reward Model and then using RL to optimize it.

What does DPO do?

- DPO eliminates the need to train a reward model, sample from the LLM during fine-tuning, or perform significant hyperparameter search.



Comparison with Baseline Models

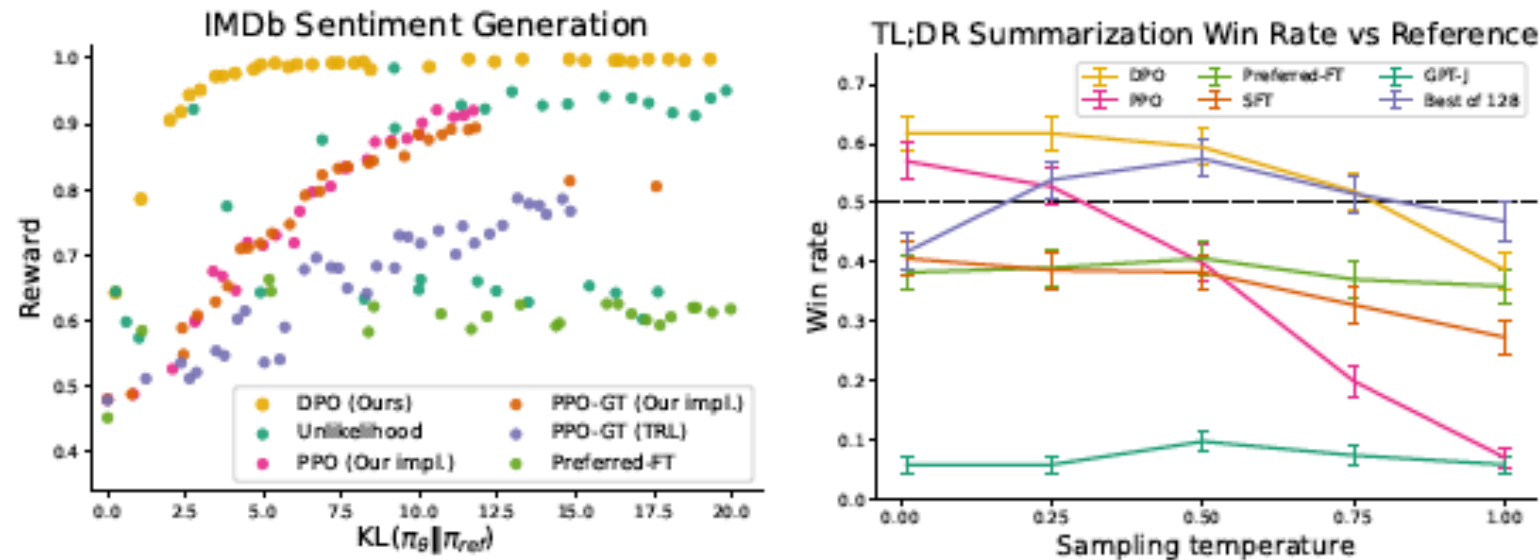
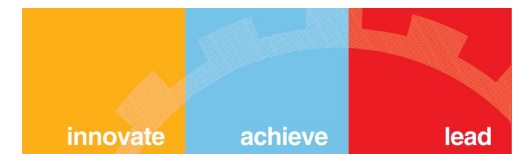


Figure 2: **Left.** The frontier of expected reward vs KL to the reference policy. DPO provides the highest expected reward for all KL values, demonstrating the quality of the optimization. **Right.** TL;DR summarization win rates vs. human-written summaries, using GPT-4 as evaluator. DPO exceeds PPO's best-case performance on summarization, while being more robust to changes in the sampling temperature.

- **Preferred-FT:** Fine-tune the model on $(y_{\{w\}})$
- **PPO-GT:** reward model is the ground truth of the sentiment
- **Unlikelihood:** optimize the policy model to maximize $(P(y_{\{w\}}))$ and minimize $(P(y_{\{l\}}))$
- **Best of N:** sampling N responses from the SFT model (very inefficient)

Comparison between PPO and DPO

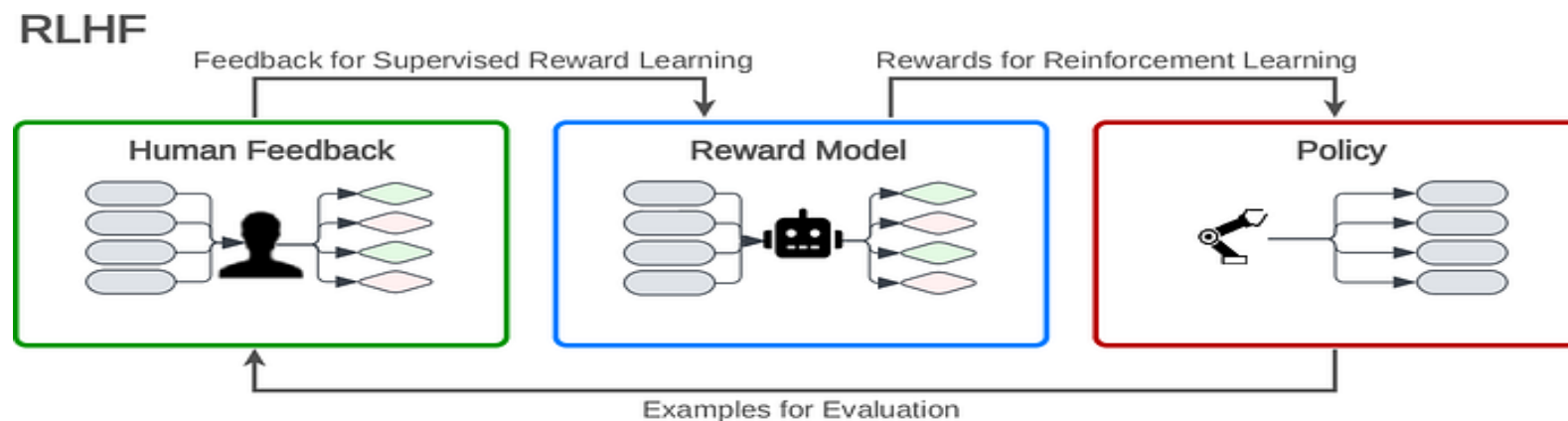
- DPO training is cheaper and more stable than PPO training
- PPO can handle more informative human feedback (e.g., numerical ratings) while DPO can only handle binary signals

Open Problems and Fundamental Limitations of RLHF

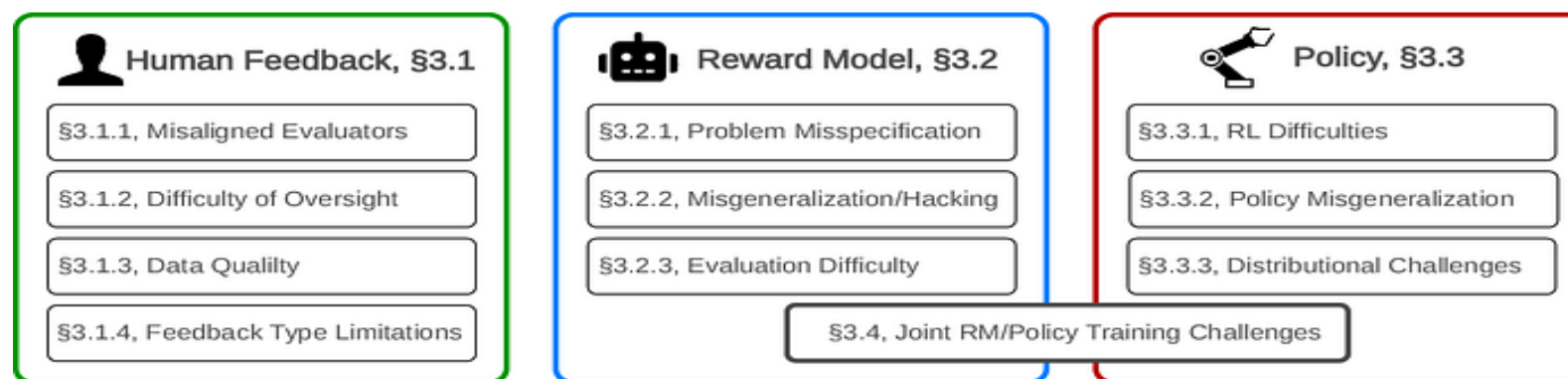
(Casper et. al, 2023)

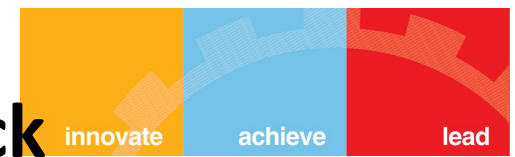


Challenges within each step: human feedback, reward model and policy



Challenges





Challenges with Obtaining Human Feedback

- Human evaluators may have biases

Studies found that ChatGPT models became politically biased post RLHF.

- Good oversight is difficult

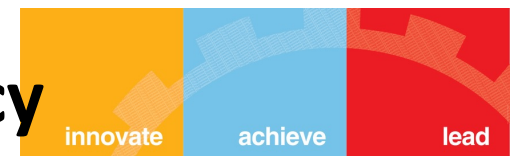
Evaluators are paid per example and may make mistakes given time constraints; poor feedback on evaluating difficult tasks

- Data quality

cost / quality tradeoff

- Tradeoff between richness and efficiency of feedback types
comparison-based feedback, scalar feedback, correction feedback, language feedback, ...

Challenges with the Reward Model and Policy



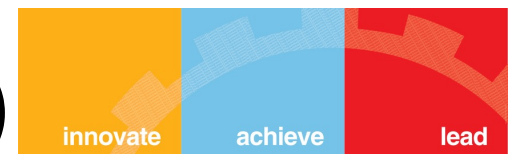
Challenges with the Reward Model

- A single reward function cannot represent a diverse society of humans
- Reward misgeneralization: reward models may fit with human preference data with unexpected features
- Evaluation of a reward model is difficult and expensive

Challenges with the Policy

- Robust reinforcement learning is difficult
 - balance between exploring new actions and exploiting known rewards
 - the challenge intensifies in high-dimensional or sparse reward settings
- Policy mis generalization: training and deployment environment is difference

Group Relative Policy Optimization (GRPO)



<https://arxiv.org/pdf/2402.03300>

What is GRPO (Group Relative Policy Optimization)? 🛠️

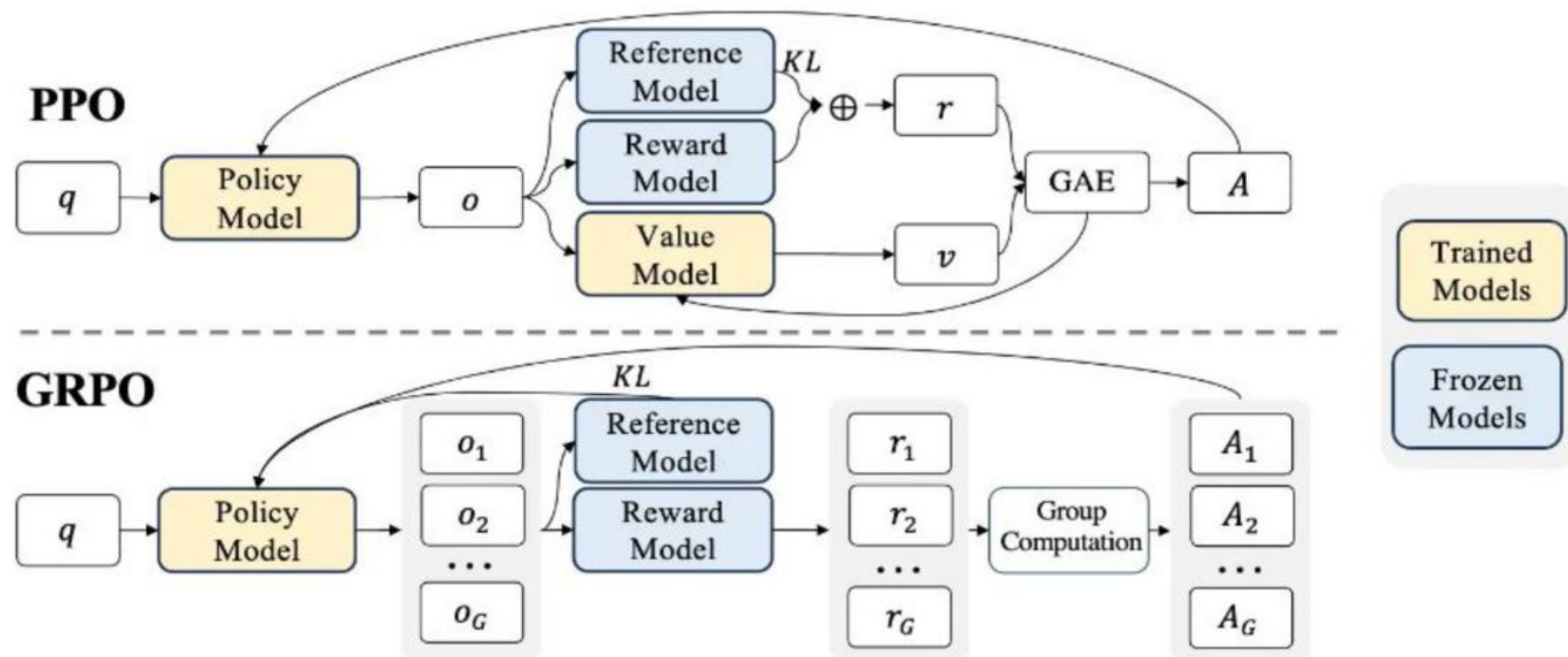


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

GPRO- Numerical



The Scenario

Imagine we are training an AI to be a helpful math tutor.

The Prompt (q): "Explain how to solve $10-2\times 3$ "

Step 1: Group Sampling (The "G" in GRPO)

Instead of generating just one answer, the Policy Model generates a **group** of 4 different answers (

a)Output 1 (o1): Multiplication first. $2\times 3=6$.

Then $10-6=4$. The answer is 4."
(Correct & Clear)

b) Output 2 (o2): "The answer is 4." (Correct, but no explanation)

c)Output 3 (o3): "You go left to right. $10-2=8$
Then $8\times 3=24$." (Wrong logic)

d)Output 4 (o4): "Math is hard." (Useless)

Step 2: Scoring (The Reward Model)

The Reward Model grades each answer on a scale of 1-10 based on accuracy and helpfulness.

- Output 1 (r1): 10** (Perfect)
- Output 2 (r2): 7** (Correct but brief)
- Output 3 (r3): 3** (Incorrect math)
- Output 4 (r4): 0** (Irrelevant)

Step 3: Group Computation (The "Relative" Part)

This is where GRPO works its magic. Instead of asking a separate "Value Model" network *"Is a score of 7 good for this specific hard math question?"*, it simply looks at the group average.

1. Calculate Average Score (Baseline): $(10+7+3+0)/4=5.0$

The baseline for this prompt is **5.0**.

2. Calculate Advantage (A):

The model compares each answer to the group average.

Advantage = Score - Average

- a) **Output 1 (A1):** $10 - 5.0 = +5.0$ (Big Winner)
- b) **Output 2 (A2):** $7 - 5.0 = +2.0$ (Small Winner)
- c) **Output 3 (A3):** $3 - 5.0 = -2.0$ (Loser)
- d) **Output 4 (A4):** $0 - 5.0 = -5.0$ (Big Loser)

Step 4: The Update (Optimization)

The model updates its brain (weights) based on these advantages:

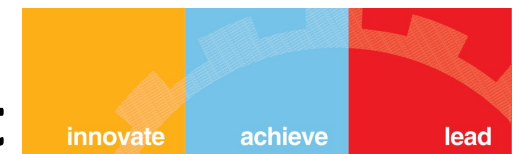
- **Strongly Reinforce Output 1:** "Do more reasoning like this!"
- **Slightly Reinforce Output 2:** "This was okay, do this if you have to."
- **Discourage Output 3:** "Stop doing math left-to-right."
- **Strongly Discourage Output 4:** "Never say 'Math is hard' when asked for help."

Comparison of PPO and GPO



Feature	PPO (Proximal Policy Optimization)	GRPO (Group Relative Policy Optimization)
Core Mechanism	Actor-Critic with clipped policy updates	Relative reward optimization using groups of generated outputs
Advantage Estimation	Learns a value function (critic)	Computes relative advantages from group rewards
Additional Network	Requires actor and critic networks	Primarily requires only the policy network
Memory/Compute Cost	Higher due to critic maintenance	Lower due to critic removal
Training Complexity	More hyperparameters and actor-critic interactions	Simpler training pipeline
Stability Source	Clipped objective prevents large policy updates	Relative reward normalization reduces variance
Typical Applications	Robotics, games, RLHF, control systems	LLM alignment, reasoning, and post-training
Main Limitation	Critic estimation errors can affect learning	Performance depends heavily on reward quality and group sampling

State of the Art (SoTA) in RLHF & LLM Alignment



Constitutional AI (Anthropic, 2022)

Trains models using a set of principles (“constitution”). AI self-critiques responses using principles, reducing reliance on human labelers via RLAIIF.

RLHF + Self-Play (Meta, 2024)

Llama 3 uses iterative RLHF with rejection sampling. Model generates responses, best ones become SFT data for next iteration.

Reward Model Ensembles

Multiple reward models trained on diverse feedback signals reduce reward hacking and improve generalization.

GRPO in DeepSeek-R1 (2025)

DeepSeek-R1 uses GRPO to train chain-of-thought reasoning without a value network, achieving o1-level math performance.

Process Reward Models (PRMs)

Rewards each reasoning step (not just the final answer). OpenAI’s PRM800K enables step-level feedback for math reasoning.

SPIN / Online DPO (2024)

Self-play fine-tuning (SPIN) and online DPO collect new model-generated data each iteration, avoiding stale offline datasets.

Key Takeaways



1. RLHF is the backbone of modern LLM alignment

Human feedback is used to shape model behavior toward helpfulness, harmlessness, and honesty.

2. PPO vs DPO: tradeoffs in stability vs simplicity

PPO is powerful but complex; DPO removes the reward model and treats alignment as classification.

3. GRPO advances reasoning without a critic network

Group-relative rewards eliminate the value model, reducing memory and enabling strong reasoning (DeepSeek-R1).

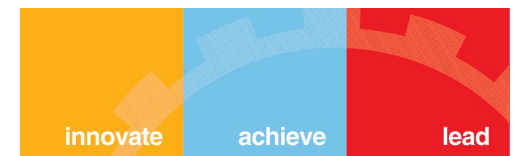
4. Human feedback has fundamental bottlenecks

Annotator bias, reward hacking, and scalability remain open problems (Casper et al., 2023).

5. The field is rapidly moving toward scalable, automated alignment

Constitutional AI, RLAIIF, and process reward models reduce human-in-the-loop dependency.

2. Parameter Efficient Fine Tuning



1

Full FT vs PEFT

Why PEFT exists & key trade-offs

2

Prompt & Prefix Tuning

Soft prompt methods (Lester 2021, Li 2021)

3

Adapter-Based Methods

Injecting small trainable modules (Houlsby 2019)

4

LoRA

Low-Rank reparametrisation (Hu 2021)

5

Quantization & QLoRA

4-bit NF4, double quant, paged optimizers (Dettmers 2023)

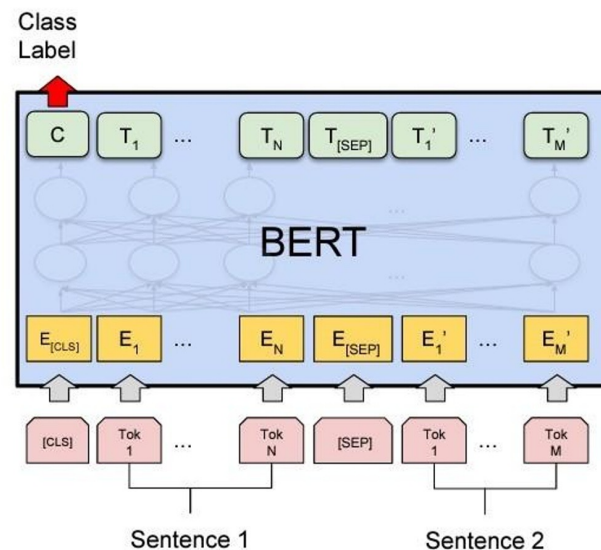
Full FT Vs PEFT



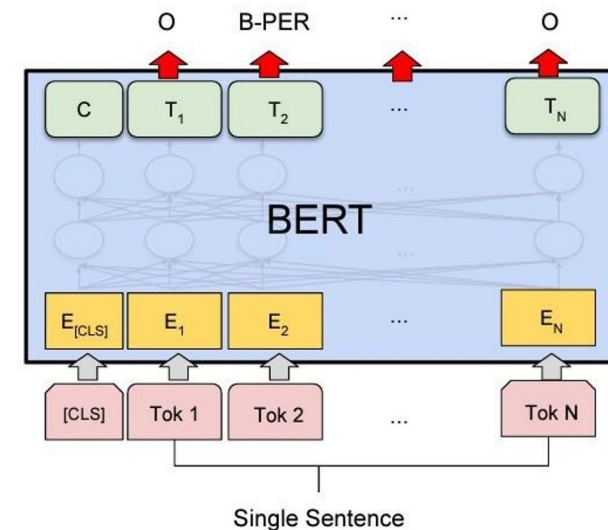
Feature	Full Fine-Tuning	PEFT (LoRA, QLoRA, etc.)
Parameters Updated	All weights ~100%	Adapters only ~0.1% - 5%
Hardware	Multi-GPU / Cluster required	Single Consumer GPU often sufficient
Storage	A full copy of the model for every task	One base model + tiny adapter files (MBs)
Catastrophic Forgetting	Higher risk (overwriting knowledge)	Low risk (base knowledge is frozen)
Best Use Case	Fundamental behavior change (e.g., teaching a model a new language or complex biological sequences)	Style/Task adaptation (e.g., summarization, classification, SQL generation)

Background: Vanilla Fine-tuning

- Attach a task-specific layer to the last layer of the pre-trained transformer output
- Update the weights of all the parameters by backpropagating gradients on a downstream task



(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Why PEFT? The Motivation

Vanilla Fine-Tuning

- Attach a task-specific layer to the pre-trained transformer output
- Back-propagate gradients through ALL layers on downstream task
- Requires storing one full model copy per task

PEFT Strategies

- Prompt-based: add tunable tokens to input
- Adapter-based: inject small modules per layer
- Reparametrisation: LoRA – decompose weight updates

**~0.1–5%
params**

trainable

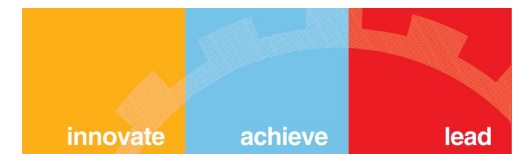
10-100×

storage savings

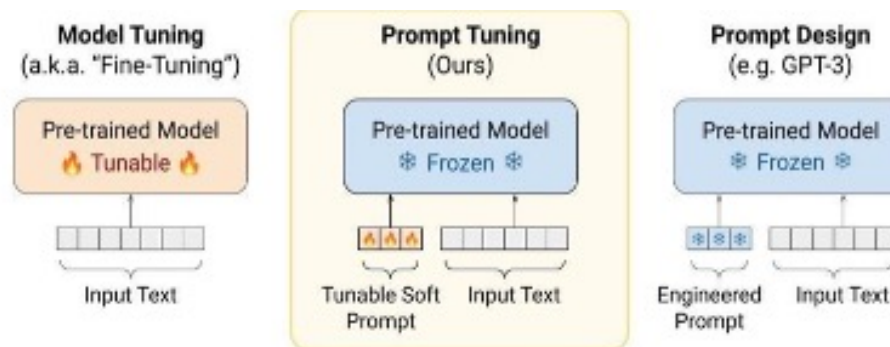
**Consumer
GPU**

sufficient

Prompt Tuning (Lester et al., 2021)



- Prepend a sequence of tokens as tunable embeddings to the input ("soft prompts")
- The entire pre-trained Transformer is FROZEN – only soft prompt vectors are trained
- Only a small set of parameters need to be stored for each task
- Prompt tuning becomes more effective as the pre-trained model becomes larger
- Larger models perform well even with a small number of prompt tokens
- Initializing with task-relevant tokens (e.g. "summarization") significantly outperforms random initialisation

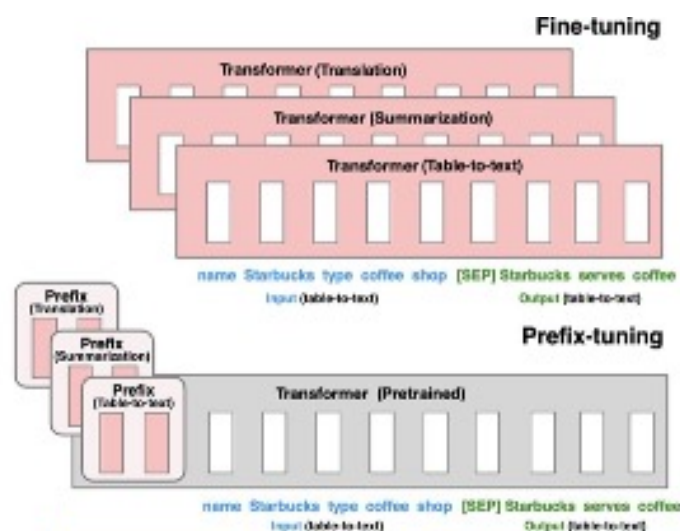


Method Comparison		
Model Tuning	All params tunable	Highest accuracy, huge cost
Prompt Tuning	Only soft prompt	Matches model-tuning at scale
Prompt Design (GPT-3)	Manual text prompts	No training; prompt-sensitive

Prefix Tuning (Li & Liang, 2021)



- Soft prompts prepended to EACH layer of the Transformer (not just input)
- Tune continuous task-specific vectors; all original weights remain frozen
- Prefix length acts as a key hyperparameter; optimal length is task-dependent
- Prefix occupies context window – reduces space available to input tokens
- Optimal length varies: ~200 tokens for summarization, ~10 for table-to-text
- Random initialisation → high variance; task-relevant words (e.g. 'summarization') greatly improve stability



Key Findings

Prefix length

~200 for sum.

Prefix length

~10 for t2t

Initialization

real words ↑↑

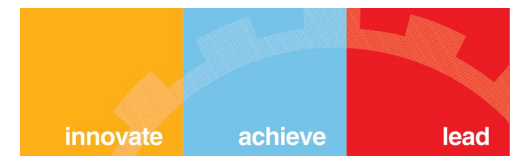
Storage

just prefix vecs

Prompt Tuning vs Prefix Tuning

Prompt Tuning: soft tokens prepended only at the input embedding layer. **Prefix Tuning:** virtual tokens prepended to every transformer layer's key/value matrices.

Issues with Prompt / Prefix Tuning



Context window overhead

The prefix occupies tokens in the fixed context window, leaving fewer tokens for real input content.

Task-specific prefix length

Optimal prefix length varies per task — requires separate tuning for each downstream application.

Initialisation sensitivity

Random initialisation leads to high variance; real-token initialisation is recommended but task-specific.

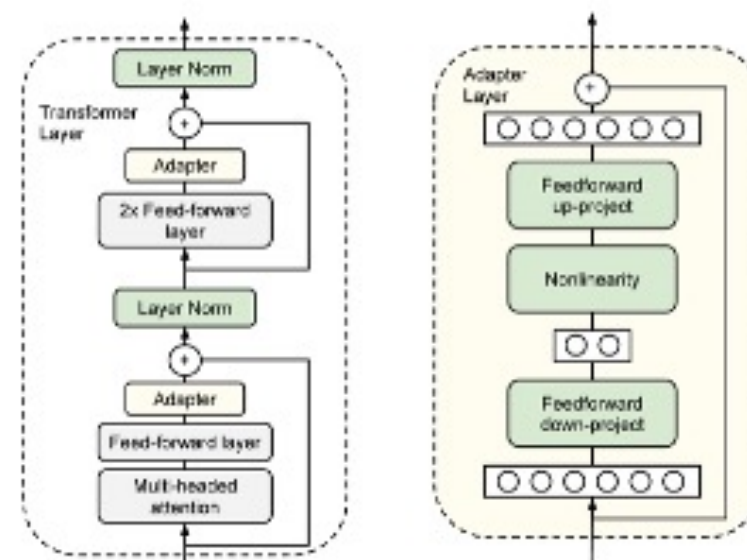
Scale dependency

Prompt tuning degrades significantly on smaller models; gains are mostly observed at very large scale (Lester et al., 2021).

Adapter-Based Methods (Houlsby et al., 2019)



- Vanilla fine-tuning can be seen as adding an extra layer on top of the Transformer
- Adapter modules perform more general architectural modifications: injecting new layers/modules into the original network
- During training, original network weights are untouched – only adapter weights update
- Two small sub-layers: down-projection $\rightarrow$ non-linearity $\rightarrow$ up-projection
- Near-identity initialisation so training stability is preserved
- Only adapter layers + final classification layer update during training
- Achieves GLUE performance within 0.4% of full fine-tuning using <2% of trainable parameters



Pros: empirically effective in multi-task settings; computationally efficient vs. full fine-tuning. Cons: adds layers $\rightarrow$ slower inference; increases model size.

LoRA: Low-Rank Adaptation (Hu et al., 2021)

Key Insight

Neural networks contain many dense layers performing matrix multiplication. Inspired by the intrinsic dimension hypothesis, the weight updates ΔW during fine-tuning have low intrinsic rank.

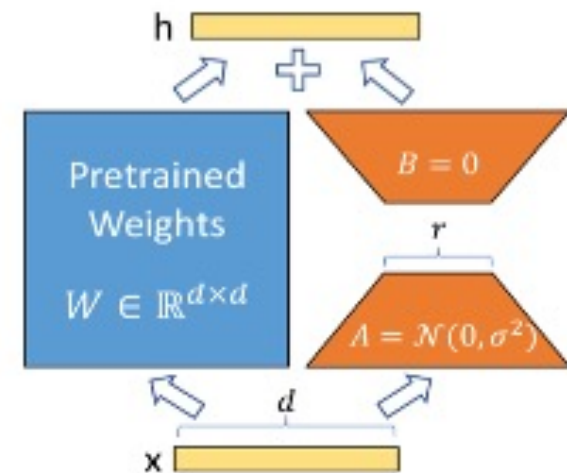
Pre-trained matrix: $W_0 \in \mathbb{R}^{(d \times k)}$ – frozen during training

Updated matrix: $W_0 + \Delta W = W_0 + BA$

Low-rank matrices: $A \in \mathbb{R}^{(r \times k)}$, $B \in \mathbb{R}^{(d \times r)}$, where $r \ll \min(d, k)$

Hidden state: $h = W_0 x \rightarrow h = W_0 x + \Delta W x = W_0 x + BAx$

Rank r : Typically 2-64; $r=4$ or $r=8$ already gives strong results



Hu, Edward J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022.

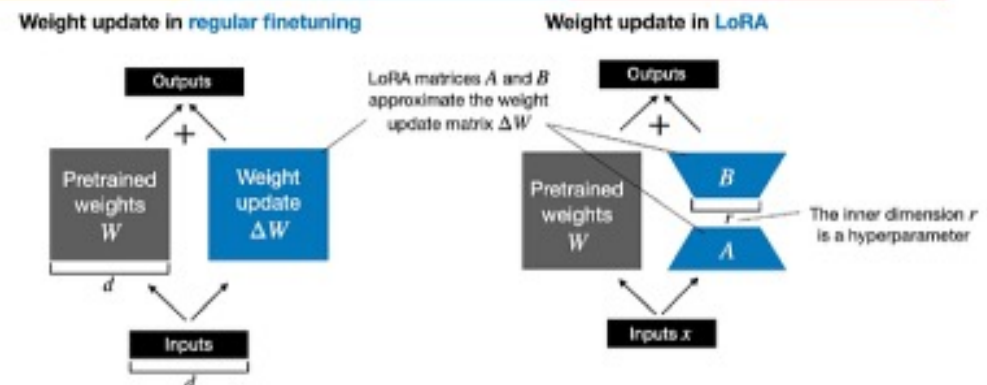
Applying LoRA to Transformers

- In principle, LoRA can be applied to any weight matrices in deep learning
- In practice, the focus is on applying LoRA to attention matrices in Transformers: W_q , W_k , W_v , W_o
- Rank r ranges from 2 to 64 — typically $r = 4$ or $r = 8$ gives strong results
- GPU VRAM requirement drops from 1.2 TB $\rightarrow$ >350 GB for large models
- Checkpoint storage: 350 GB (full) $\rightarrow$ 35 MB (LoRA adapter) — 10,000 $\times$ smaller

Which Matrices? (Ablation — Hu et al., 2021)

Applying LoRA to both W_q and W_v (attention query + value matrices) gives the best trade-off between parameter count and accuracy. $r = 4$ already gives strong results; $r = 8$ covers most meaningful subspaces for most tasks.

LoRA (GPT-3, $r=4$): WikiSQL 73.8 | MNLI 91.0 | SAMSum 53.0 — 37M trainable params vs 175B total (Hu et al., 2021 Table 5)



Hu, Edward J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022.

LoRA: Comparison with Full FT & Adapters

Full Fine-Tuning	Adapters (Houlsby 2019)	LoRA (Hu 2021)
• Updates ALL weights • High VRAM (multi-GPU) • Separate model per task • Risk of catastrophic forgetting • Best absolute accuracy	• New layers injected • Slower inference (new ops) • Near-zero init stability • Effective multi-task • Small overhead at inference	• No new layers at inference • Merge BA into $W_0 \rightarrow$ zero latency • $\sim 0.1\%$ trainable params • 10000$\times$ smaller checkpoints • Matches FT on most benchmarks

Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum
		Acc. (%)	Acc. (%)	R1/R2/RL
GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5
GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5
GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5
GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5
GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8
GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1
GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9
GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1

- LoRA outperforms several baselines with comparable or fewer trainable parameters.

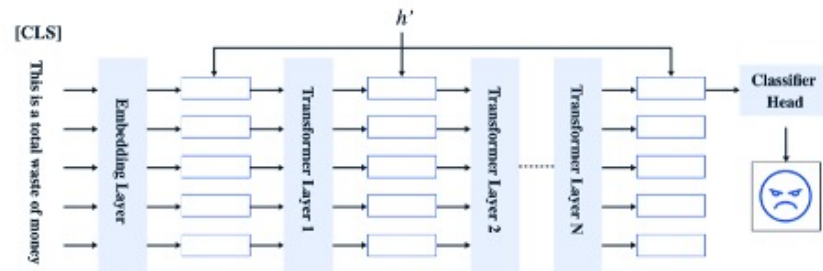
PEFT Methods Taxonomy

Addition-Based	Reparametrisation-Based	Selection-Based
- Adapters (Houlsby 2019) - Prompt Tuning (Lester 2021) - Prefix Tuning (Li 2021) - BitFit (IA)³	- LoRA (Hu 2021) - QLoRA (Dettmers 2023) - KronA - FourierFT	- Diff-Pruning - Fish-Mask - Lottery Ticket
<i>Add new parameters while keeping original weights frozen</i>	<i>Decompose weight updates into compact low-rank form</i>	<i>Select a sparse subset of existing parameters to update</i>

Source: Lialin et al. (2023) "Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning"

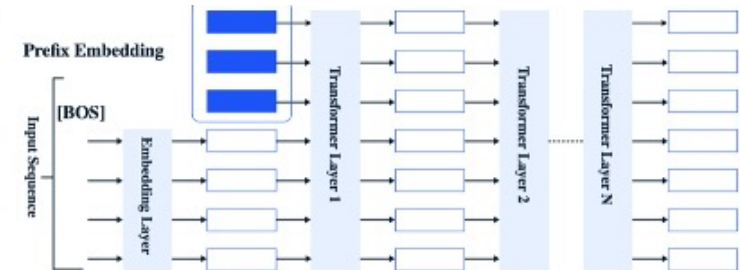
Summary

• Vanilla Fine-Tuning



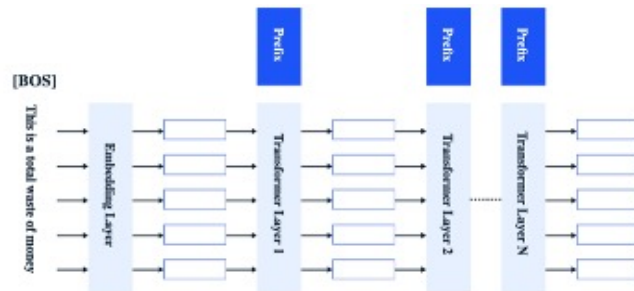
Source: <https://www.leewayhertz.com/parameter-efficient-fine-tuning/>

• Prompt-Tuning



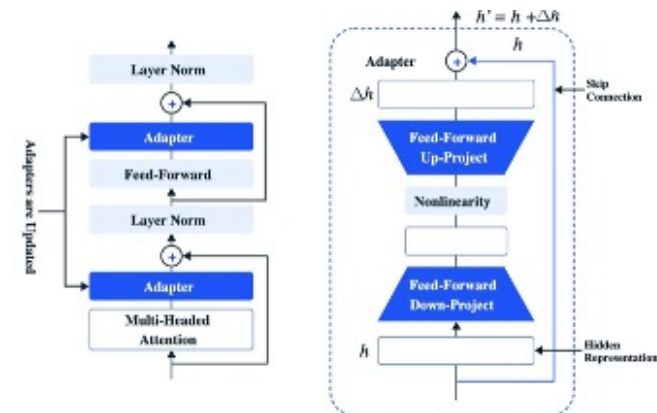
```
def prompt_tuning_attention(input_ids):
    q = x @ W_q
    k = cat([s_k, x]) @ W_k # prepend a
    v = cat([s_v, x]) @ W_v # soft prompt
    return softmax(q @ k.T) @ v
```

• Prefix-Tuning

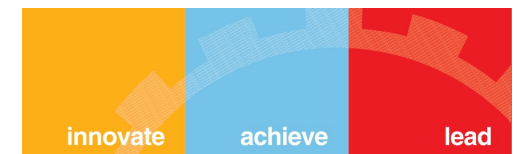


```
def transformer_block_for_prefix_tuning(x):
    soft_prompt = FFN(soft_prompt)
    x = concat([soft_prompt, x], dim=seq)
    return transformer_block(x)
```

• Adapters



Quantization Basics



Motivation: 32-bit floating-point computations are memory-intensive. Quantization reduces the precision of numbers used in the model, dramatically cutting memory.

int8 tensor

Range: $[-127, 127]$ (16 levels)

int4 tensor

Range: $[-8, 7]$ (16 levels, signed)

float32

Range: $\sim 1.18 \times 10^{-38}$ to 3.4×10^{38}

Quantization / Dequantization Formulas

Quantize: $X[\text{int8}] = \text{round} \left(\left(127 / \text{absmax}(X[\text{FP32}]) \right) \times X[\text{FP32}] \right)$

Dequantize: $\text{dequant}(X[\text{FP32}], X[\text{int8}]) = X[\text{int8}] / \left(127 / \text{absmax}(X[\text{FP32}]) \right)$

⚠ *Quantization with outliers leads to larger quantization errors.*

4-bit NormalFloat (NF4) (Dettmers et al., 2023)



- Neural network weights typically follow a zero-centered normal distribution $N(0, \sigma^2)$
- NF4 is an information-theoretically optimal quantization type for normally distributed data
- Normalization: transform all weights to a single fixed distribution by scaling with σ and setting range to $[-1, 1]$ with mean 0 and standard deviation 1
- Quantize the normalized weights to 4 bits — 16 levels evenly spaced within the normalized values
- NF4 yields better empirical results than both 4-bit integers and 4-bit floats
- Small dequantization error: for example, 0.1832 rounds to 0.2 $\rightarrow$ error ≈ 0.0168

NF4 (NormalFloat)

Information-theoretically optimal for normal distributions. Used in QLoRA.

int4 (Integer)

Uniformly spaced integers. Not optimal when weight distribution has thin tails.

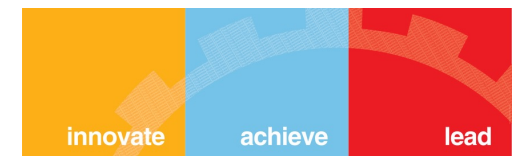
fp4 (Float)

4-bit floats — fewer levels near zero where weights cluster.

Dettmers, Tim, et al. "QLoRA: Efficient Finetuning of Quantized LLMs." *NeurIPS 2023*. <https://doi.org/10.48550/arXiv.2305.14314>

QLoRA: Quantization + Low-Rank Adapters

(Dettmers et al., 2023)



QLoRA combines 4-bit NF4 quantization with Low-Rank Adapters to enable efficient fine-tuning of large language models on a single consumer GPU.

1. 4-bit NormalFloat

Pre-trained model weights quantized to NF4 and frozen. Significantly reduces memory usage at load time.

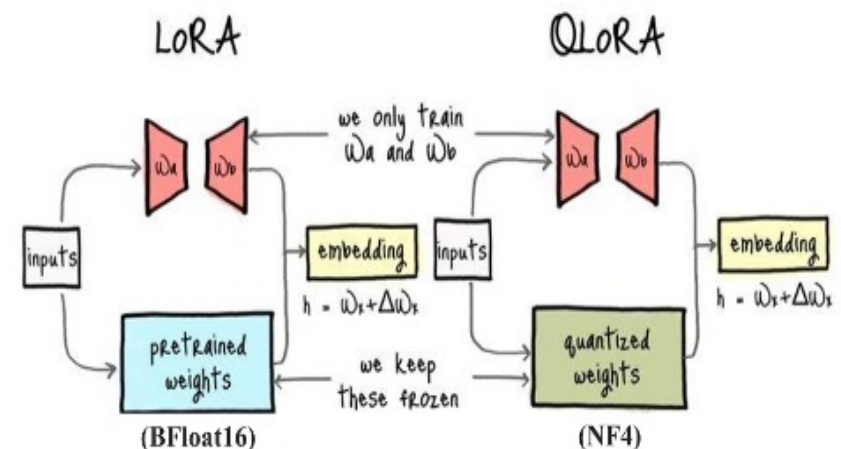
2. Double Quantization

Quantize the quantization constants themselves, saving ~ 0.37 bits/parameter (≈ 3 GB for a 65B model).

3. Paged Optimizers

NVIDIA unified memory to avoid gradient checkpointing spikes when processing long-sequence mini-batches.

Memory reduction: Full FT of 65B model requires >780 GB GPU memory. QLoRA trains same model in **<48 GB** without degrading performance vs. 16-bit fully fine-tuned baseline.



QLoRA Architecture & Training Flow



1

Load Base Model

Load pre-trained LLM (e.g. LLaMA 70B) in 4-bit NF4 format. Weights are frozen — no gradient flows through them.

2

Add LoRA Adapters

Attach small trainable rank-r matrices A and B to each transformer layer (attention W_q , W_v). Adapters are in full 32-bit precision.

3

Forward Pass

Input passes through quantized (frozen) base weights + full-precision LoRA adapters. Output = $W_o x + B A x$.

4

Backward Pass

Gradients flow only through LoRA matrices A and B. Quantized weights dequantized temporarily for computation; requantized after.

5

Final Model

Consists of original 4-bit weights + trained adapter matrices in bfloat16/fp32. Can merge adapters into base model for zero-latency inference.

Dettmers, Tim, et al. "QLoRA: Efficient Finetuning of Quantized LLMs." NeurIPS 2023.



Memory Requirements for LLM Serving & Training

GPU Memory (inference): $M = (P \times B) / Q \times 1.2$ where P = parameters, B = bytes per param, Q = quantization divisor

LLaMA 70B Examples (at 16-bit)		
70B model, float32	$70 \times 10^9 \times 4 \text{ bytes} \times 1.2$	$\approx 336 \text{ GB}$
70B model, bfloat16	$70 \times 10^9 \times 2 \text{ bytes} \times 1.2$	$\approx 168 \text{ GB}$ (2×A100 80GB needed)
70B model, int8	$70 \times 10^9 \times 1 \text{ byte} \times 1.2$	$\approx 84 \text{ GB}$
70B model, int4/NF4	$70 \times 10^9 \times 0.5 \text{ bytes} \times 1.2$	$\approx 42 \text{ GB}$ (fits on 1×A100 80GB)

Meta-Llama-3-8B-Instruct Training Memory (from HuggingFace)			
Method	float32	int8	int4
Total Size (GB)	27.96	13.98	6.99 / 3.49
Training using Adam (GB)	113.83	55.92	27.96 / 13.98
LoRA Fine-Tuning (GB)	34.99	18.22	9.63 / 5.63

Memory formula from Dettmers et al. NeurIPS 2023; training table from <https://huggingface.co/spaces/Vokturz/can-it-run-llm>

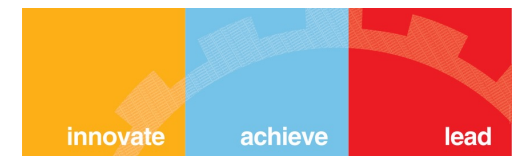
Comparison of PEFT Categories

	Adapters	Soft Prompts	Structured Sparse	Reparameterisation	Hybrid
Storage efficiency	✓	✓	✓	✓	✓
Training	✓	✓	X*	✓	✓
RAM Speed	X*	✓	✓	✓	X*
Single-task	X*	X*	✓	✓	✓
Multi-task	✓	X*	X*	X*	✓
Efficient inference	X*	✓	✓	✓	X*

✓ = effective | X* = might vary based on specific method | Table adapted from Lialin et al. (2023)

Implemented in: HuggingFace PEFT library (peft.huggingface.co) and AdapterHub (adapterhub.ml)

Summary



Full Fine-Tuning

Updates all weights. Highest accuracy but requires multi-GPU, per-task storage, and risks catastrophic forgetting.

Prompt / Prefix Tuning

Soft tokens added to input or every layer. Lester 2021 (input only), Li 2021 (every layer). Context window cost.

Adapters

Inject trainable sub-layers (down→non-linear→up) into each transformer block. Near-FT performance with ~2% params. Houlsby 2019.

LoRA

Reparametrize $\Delta W = BA$ with rank $r \ll d$. Merge at inference → zero latency. 10 000× smaller checkpoints. Hu 2021.

QLoRA

4-bit NF4 quantization + LoRA adapters in 32-bit. Double quantization. Paged optimizers. 65B model on <48 GB VRAM. Dettmers 2023.

PEFT is now the default paradigm for adapting large language models to downstream tasks at low cost.

Key Takeaways



PEFT is now the default paradigm

Adapting large pre-trained models without retraining all weights is the practical standard — enabling downstream tasks on consumer hardware.

Rank r is the core design choice in LoRA

$r = 4$ or 8 covers most meaningful subspaces. Merging BA into W_o at inference yields zero added latency — a key advantage over Adapters.

QLoRA democratizes 65B+ model fine-tuning

NF4 + double quantization + paged optimizers reduce a 65B fine-tune from >780 GB to <48 GB — single A100 territory.

Choose method by inference constraint

Prompt/Prefix tuning adds context window cost; Adapters add latency; LoRA/QLoRA add neither at inference after merging.

Soft-prompt methods are model-scale dependent

Prompt tuning matches full fine-tuning only at very large scale (Lester 2021). Prefix tuning extends coverage to every layer but consumes context budget.

Tiny adapters enable massive storage savings

One shared base model + MB-sized adapter files per task replaces storing separate full-model copies, enabling practical multi-task deployment.

The goal: match full fine-tuning accuracy at a fraction of the compute, memory, and storage cost.

References



-
- [1] Wang, Y. et al. "Self-Instruct: Aligning Language Models with Self-Generated Instructions." ACL 2023.
 - [2] Ouyang, L. et al. "Training language models to follow instructions with human feedback." NeurIPS 2022.
 - [3] Dettmers, T. et al. "QLoRA: Efficient Finetuning of Quantized LLMs." NeurIPS 2023. arXiv:2305.14314
 - [4] Rafailov, R. et al. "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." NeurIPS 2023.
 - [5] Hu, E. J. et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR 2022. arXiv:2106.09685
 - [6] Housby, N. et al. "Parameter-Efficient Transfer Learning for NLP." ICML 2019.
 - [7] Lester, B., Al-Rfou, R., & Constant, N. "The Power of Scale for Parameter-Efficient Prompt Tuning." EMNLP 2021.
 - [8] Li, X. L., & Liang, P. "Prefix-Tuning: Optimizing Continuous Prompts for Generation." ACL 2021.
 - [9] Lialin, V. et al. "Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning." arXiv 2023.
 - [10] Schulman et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
 - [11] Shao et al. (2024). DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models (GRPO). arXiv:2402.03300.
 - [12] Casper et al. (2023). Open Problems and Fundamental Limitations of RLHF. arXiv:2307.15217.

SoTA & Supplementary

- [13] Bai et al. (2022). Constitutional AI: Harmlessness from AI Feedback. Anthropic. arXiv:2212.08073.
- [14] Lightman et al. (2023). Let's Verify Step by Step (Process Reward Models). arXiv:2305.20050.
- [15] Chen et al. (2024). SPIN: Self-Play Fine-Tuning Converts Weak Language Models to Strong Language Models. ICML 2024.
- [16] DeepSeek AI (2025). DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948.
- [17] Meta AI (2024). The Llama 3 Herd of Models. arXiv:2407.21783.